

Agents & Tools

Stop the model from only talking and let it *act*: give it tools, and let it decide which to call and when. Built with the modern `create_agent` API — shown for both **Ollama** and the **Gemini API**.

THIS VOLUME COVERS

Phase 4 — Agents & Tools
Project: a web-search agent

From chains to agents

Every chain you have built so far follows a fixed path you designed: prompt → model → parser, always in that order. An **agent** is different. You give the model a set of **tools** and a goal, and the *model itself* decides which tools to call, in what order, and when it is finished.

That decision-making happens in a loop: the model looks at the conversation, optionally asks to call a tool, reads the tool's result, and then decides whether to call another tool or give a final answer. This is what lets an assistant search the web, run a calculation, or query a database on its own initiative — behaviour you could not script in advance because it depends on the question.

THE AGENT LOOP



! The big 1.0 change — read this

Pre-1.0 tutorials build agents with `initialize_agent`, `AgentExecutor`, and `create_react_agent` plus an `agent_scratchpad` prompt. Those are legacy. The current, supported way is a single function, `create_agent`, from `langchain.agents`. It is built on **LangGraph**, which gives agents durable execution, streaming, and memory for free — and you do not need to learn LangGraph to use it.

PHASE 4

Agents & Tools

We start by writing tools with the `@tool` decorator, then assemble them into an agent with `create_agent`, watch the agent reason through a problem, plug in real web search with Tavily, and give the agent memory. The project is a research assistant that searches the web to answer current questions.

4.1 Tools: giving the model abilities

A **tool** is just a Python function the model is allowed to call. The simplest way to make one is the `@tool` decorator from `langchain_core.tools`. LangChain reads the function's **name**, its **type hints**, and — crucially — its **docstring** to build a description the model uses to decide when the tool is relevant.

```
PYTHON

from langchain_core.tools import tool

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers and return the product."""
    return a * b

@tool
def current_date() -> str:
    """Return today's date in YYYY-MM-DD format."""
    import datetime
    return datetime.date.today().isoformat()
```

Each decorated function is now a tool object you can inspect and even call directly:

```
PYTHON

print(multiply.name)           # multiply
print(multiply.description)    # Multiply two numbers and return the product.
print(multiply.invoke({"a": 6, "b": 7})) # 42.0
```

* The docstring is not optional

The model never sees your code — only the name, arguments, and docstring. A vague or missing docstring is the number-one reason an agent fails to call a tool, or calls the wrong one. Write the docstring as if it were documentation for a colleague: say plainly what the tool does and when to use it.

4.2 Building an agent with `create_agent`

`create_agent` wires a model and a list of tools into the reasoning loop from page one. You give it three things: a model, the tools, and an optional `system_prompt` that sets the agent's behaviour. The model can be a "provider:model" string (just like `init_chat_model`) or a model instance.

```
PYTHON

from dotenv import load_dotenv
from langchain.agents import create_agent

load_dotenv()

# --- choose your engine -----
MODEL = "ollama:llama3.1"          # local (must support tools)
# MODEL = "google_genai:gemin-2.5-flash"  # cloud
# -----

agent = create_agent(
    model=MODEL,
    tools=[multiply, current_date],
    system_prompt="You are a helpful assistant. Use the tools when they help.",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What is 23.5 times 8, and what is today's date?"}]}
)

print(result["messages"][-1].content)
```

Two things to notice about the call. You pass input as a `messages` list (the same role/content shape from Part 1). And the return value is a dict whose `messages` key holds the *entire* trace — your question, the model's tool calls, each tool's result, and the final answer. The finished reply is always the last message, hence `result["messages"][-1].content`.

! Agents need tool-capable models

Tool calling is a model feature, not something LangChain can fake. Gemini models support it strongly. Among local Ollama models, pick one that advertises tool support such as `llama3.1` — smaller or older models may ignore tools or call them unreliably. For serious agent work, cloud models are the safer choice; for private, local experiments, `llama3.1` is a fine start.

4.3 Watching the agent reason

When something goes wrong, you want to see *why* the agent did what it did. Instead of `invoke`, use `stream` with `stream_mode="updates"` to watch each step as it happens — every tool call and every tool result appears in real time.

```
PYTHON

for step in agent.stream(
    {"messages": [{"role": "user", "content": "What is 19 * 23?"}]},
    stream_mode="updates",
):
    print(step)
    print("---")
```

You will see the model first emit a request to call `multiply` with `{"a": 19, "b": 23}`, then a tool result of `437`, then the model's final sentence. Seeing this trace is the fastest way to debug a misbehaving agent — and in Part 5 we capture exactly this trace automatically with LangSmith.

The agent decides — you do not

If you ask “what is the capital of France?” the agent will likely answer directly without touching any tool, because no tool helps. That autonomy is the whole point: you supply capabilities, the model supplies judgement about when to use them.

4.4 A real tool: web search with Tavily

Custom tools are great, but the classic agent ability is searching the web for current information the model could not know. The search tool recommended by LangChain is **Tavily**, which is built for LLMs. It needs its own free API key (get one at tavily.com) added to your `.env` as `TAVILY_API_KEY`.

TERMINAL

```
pip install -U langchain-tavily
```

PYTHON

```
from langchain_tavily import TavilySearch

# Reads TAVILY_API_KEY from the environment
search = TavilySearch(max_results=5)

agent = create_agent(
    model=MODEL,
    tools=[search],
    system_prompt=(
        "You are a research assistant. Use web search for current or factual "
        "questions, and cite the sources you used."
    ),
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What are the latest LangChain releases?"}]}
)
print(result["messages"][-1].content)
```

Adding a tool is literally appending to the `tools` list. Everything else — the loop, the message handling — is unchanged. That composability is why agents scale from one tool to dozens.

+ Inject today's date for current-events questions

Models do not know what “today” is. For time-sensitive research, put the current date into the `system_prompt` (or expose the `current_date` tool from page one) so the agent searches with the right timeframe instead of guessing the year.

4.5 Giving the agent memory

In Part 2 we promised that the production-grade way to add memory was LangGraph's persistence. Because `create_agent` is built on LangGraph, you get it by passing a **checkpointer**. The agent then remembers each conversation, keyed by a `thread_id` you supply in the config — the same per-session pattern as before, now handled for you.

```
PYTHON

from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model=MODEL,
    tools=[multiply, current_date],
    checkpointer=InMemorySaver(),      # remembers across turns
)

cfg = {"configurable": {"thread_id": "user-1"}}

agent.invoke({"messages": [{"role": "user", "content": "My name is Sam."}]}, config=cfg)
out = agent.invoke({"messages": [{"role": "user", "content": "What is my name?"}]},
    config=cfg)
print(out["messages"][-1].content)    # -> Sam
```

When you outgrow `create_agent`

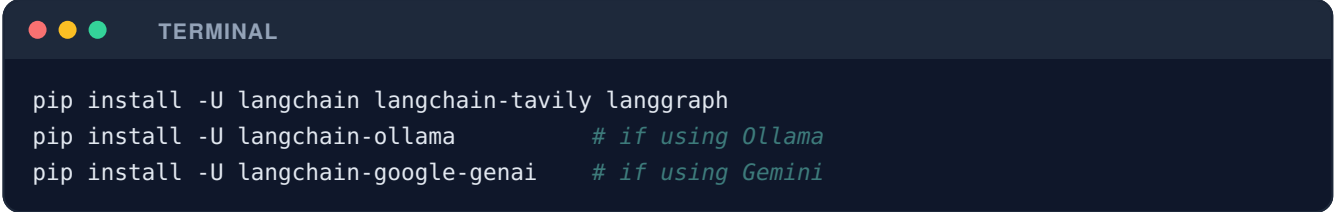
`create_agent` gives you one well-designed loop. When you need something more elaborate — multiple cooperating agents, branching workflows, human approval steps, or strict control over each transition — you drop down to **LangGraph** directly and build the graph yourself. The two compose: an agent made with `create_agent` can be a single node inside a larger LangGraph. For most applications, though, the prebuilt agent is all you need, which is why this guide stays at that level.

i InMemorySaver is for one process

As its name says, `InMemorySaver` keeps history in RAM and forgets on restart — perfect for development. For real deployments, swap in a persistent checkpointer (for example a SQLite- or Postgres-backed one) so conversations survive restarts. The agent code does not change; only the checkpointer does.

Build a web-search research agent

The project combines everything: a real web-search tool, two custom tools, memory, and a clear system prompt — wrapped in a command-line loop. Ask it current-events questions and it searches; ask it to compute and it uses the math tool; refer back to earlier turns and it remembers.



```
pip install -U langchain langchain-tavily langgraph
pip install -U langchain-ollama           # if using Ollama
pip install -U langchain-google-genai     # if using Gemini
```



```
# research_agent.py
import datetime
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_tavily import TavilySearch
from langgraph.checkpoint.memory import InMemorySaver

load_dotenv()

# --- choose your engine -----
MODEL = "ollama:llama3.1" # local (must support tools)
# MODEL = "google_genai:gemin-2.5-flash" # cloud
# -----

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers and return the product."""
    return a * b

@tool
def current_date() -> str:
    """Return today's date in YYYY-MM-DD format."""
    return datetime.date.today().isoformat()

search = TavilySearch(max_results=5) # needs TAVILY_API_KEY

agent = create_agent(
    model=MODEL,
    tools=[search, multiply, current_date],
    system_prompt=(
        "You are a concise research assistant. Use web search for current or "
        "factual questions and cite the sources you used. Use the math and date "
        "tools when relevant. If you are unsure, say so rather than guessing."
    ),
    checkpoint=InMemorySaver(), # remembers the conversation
)

config = {"configurable": {"thread_id": "session-1"}}

print("Research agent ready. Ask anything, or 'quit' to exit.\n")
while True:
    q = input("you > ").strip()
    if q.lower() in {"quit", "exit"}:
        break
    if not q:
        continue
    result = agent.invoke(
        {"messages": [{"role": "user", "content": q}]}, config=config
    )
    print("bot >", result["messages"][-1].content, "\n")
```

Run it and try a mix of question types:

● ● ● TERMINAL

```
python research_agent.py
```

● ● ● OUTPUT

```
Research agent ready. Ask anything, or 'quit' to exit.
```

```
you > Who is the current CEO of OpenAI, and what's 1995 times 12?
```

```
bot > The current CEO of OpenAI is Sam Altman (source: openai.com). And  
1995 x 12 = 23,940.
```

```
you > What was the first thing I asked you?
```

```
bot > You asked who the current CEO of OpenAI is and what 1995 times 12 equals.
```

```
you > quit
```

Make it your own

(1) Add a custom tool that reads a local file so the agent can answer from your notes *and* the web. (2) Switch the engine string and compare how reliably each model picks tools. (3) Use `agent.stream(..., stream_mode= "updates")` to print which tool the agent chose for each question. Each touches one Phase 4 idea.

Part 4 recap & what comes next

Your applications can now take actions in the world, not just produce text — the leap from assistant to agent.

You can now

- Write tools with `@tool` and good docstrings.
- Assemble an agent with `create_agent` and run it.
- Stream and read an agent's step-by-step reasoning.
- Add real web search with Tavily.
- Give an agent memory with a checkpoint, and know the LangGraph path.

Key objects introduced

- `@tool` decorator · `.name` , `.description`
- `create_agent(model, tools, system_prompt)`
- `agent.invoke({"messages": [...]})`
- `stream_mode="updates"`
- `TavilySearch` · `InMemorySaver`

★ Coming in Part 5 — Production

The final part takes your work from your laptop to the world. We add **LangSmith** to trace every step of a chain or agent so you can see exactly what happened and why; revisit **streaming** for responsive interfaces; and serve a chain or agent behind a **FastAPI** endpoint so other applications can call it — the capstone deployment, with both engines.

End of Part 4 · The LangChain Practitioner's Guide